

EFREI Paris — Panthéon-Assas Université

SA5 : Sécurité Offensive avec Python

Bruteforce, Dictionnaire, Keylogger & Ransomware

Proposé par Souheib Yousfi
Tuteur — EFREI Paris Panthéon-Assas Université

Environnement : Ubuntu | VSCode | Python 3 | Niveau : I2, Master | Durée : 3h

Chaine Youtube : [Tutoriels cybersécurité - Souheib Yousfi](#)

AVERTISSEMENT LÉGAL ET ÉTHIQUE

Les techniques présentées dans ce SA ne doivent être utilisées que dans des environnements autorisés et à des fins pédagogiques.

Toute utilisation sur des systèmes sans autorisation explicite est illégale et passible de poursuites pénales (Code pénal art. 323-1 à 323-8, peines pouvant aller jusqu'à 7 ans d'emprisonnement).

Travaillez UNIQUEMENT sur votre VM de lab.

0. Introduction & Objectifs

Ce SA explore la sécurité offensive — non pas pour former des attaquants, mais pour vous faire comprendre les techniques utilisées par les adversaires afin de mieux les contrer et s'en défendre.

Objectifs pédagogiques

- Comprendre et créer des wordlists avec Crunch et CeWL
- Implémenter et tester une attaque par dictionnaire en Python
- Implémenter et comprendre le bruteforce avec limitation de temps
- Analyser le fonctionnement d'un keylogger
- Comprendre le mécanisme d'un ransomware (simulation contrôlée)
- Apprendre les contre-mesures pour chaque type d'attaque

Packages Python requis dans ce SA

Les scripts installent automatiquement leurs dépendances au démarrage.
Si l'installation automatique échoue, ouvrez un terminal VSCode et tapez :

```
pip install pynput --break-system-packages      # keylogger
pip install pycryptodome --break-system-packages # ransomware
```

VSCode est déjà installé.

Lab 1 — Création de Wordlists

Une wordlist (liste de mots) est la base de toute attaque par dictionnaire. La qualité de la wordlist détermine le succès ou l'échec d'une attaque.

1.1 Crunch — Génération par règles

Crunch génère des wordlists selon des règles précises : longueur, charset, préfixes/suffixes.

Installation

```
sudo apt install crunch -y
```

Utilisation de base

```
# Syntaxe : crunch <min> <max> <charset> [options]

# Générer tous les mots de 4 caractères (a-z)
crunch 4 4 abcdefghijklmnopqrstuvwxyz

# Générer des numéros de téléphone français (06xxxxxxx)
crunch 10 10 0123456789 -t 06XXXXXXXX -o mon_dictionnaire.txt

# Générer des mots de passe 6-8 chars (a-z + 0-9)
crunch 6 8 abcdefghijklmnopqrstuvwxyz0123456789 -o wordlist_alphanum.txt

# Vérifier la taille générée
wc -l mon_dictionnaire.txt
head -8 mon_dictionnaire.txt
```

Option	Description	Exemple
-t <pattern>	Motif fixe (@ = minuscule, % = chiffre, ^ = symbole)	crunch 8 8 -t efrei@@@%
-o <file>	Sauvegarder dans un fichier	-o wordlist.txt
-s <start>	Commencer à un mot précis	-s aaa000
-b <size>	Découper en fichiers de taille max	-b 50mb
-l	Traiter les caractères spéciaux littéralement	-l @%^

1.2 CeWL — Wordlist depuis un site web

CeWL (Custom Word List generator) crawle un site web et extrait tous les mots pour créer une wordlist contextualisée.

Installation

```
sudo apt install cewl -y
```

Utilisation

```
# Crawler efrei.fr : profondeur 2, mots de 4+ lettres
cewl -w wordlist_efrei.txt -d 2 -m 4 https://www.efrei.fr
```

```
# Avec les emails
cewl -w wordlist_efrei.txt -d 2 -m 4 -e https://www.efrei.fr

# Avec les métadonnées des documents
cewl -w wordlist_meta.txt -d 1 -m 6 --meta https://www.efrei.fr

# Afficher les premiers mots extraits
head -12 wordlist_efrei.txt
```

1.3 Wordlist manuelle et rockyou

```
# Créer une wordlist manuellement
cat > wordlist.txt << EOF
password
123456
efrei2025
admin
qwerty
EOF

# Utilisez rockyou (wordlist célèbre, 14M mots)
ls /usr/share/wordlists/
```

Questions

- Q1. Générez avec Crunch une wordlist de numéros de téléphone 07XXXXXXXX. Combien de lignes ?
- Q2. Utilisez CeWL sur efrei.fr avec profondeur 1 et mots de 5+ lettres. Combien de mots ?
- Q3. Quelle est la différence entre une attaque par dictionnaire et une attaque par bruteforce ?
- Q4. Pourquoi les wordlists contextualisées (CeWL) sont-elles plus efficaces que les listes génériques ?

Lab 2 — Bruteforce & Attaque par Dictionnaire

⚠️ AVERTISSEMENT LÉGAL ET ÉTHIQUE

Ces techniques ne doivent être testées que sur vos propres systèmes ou dans un environnement de lab autorisé. NE JAMAIS tester sur des systèmes tiers sans autorisation écrite explicite.

2.1 Attaque par dictionnaire en Python

L'attaque par dictionnaire teste séquentiellement tous les mots d'une liste et compare leur hash au hash cible.

Script : **SA5_01_bruteforce_dictionnaire.py**

```
import hashlib

def sha256(texte):
    return hashlib.sha256(texte.encode()).hexdigest()

def attaque_dictionnaire(hash_cible, fichier_wordlist):
    with open(fichier_wordlist, 'r') as f:
        for ligne in f:
            mot = ligne.strip()
            if sha256(mot) == hash_cible:
                print(f'[✓] Trouvé : {mot}')
                return mot
    print('[✗] Non trouvé dans la wordlist.')

# Exemple :
hash_cible = sha256('efrei2025')
attaque_dictionnaire(hash_cible, 'wordlist.txt')
```

2.2 Attaque brute force

Le bruteforce génère toutes les combinaisons possibles. Extrêmement lent — efficace uniquement sur des mots de passe très courts.

```
import hashlib, itertools, string, time

def brute_force(hash_cible, longueur_max=4):
    charset = string.ascii_lowercase + string.digits
    debut = time.time()
    tentatives = 0

    for longueur in range(1, longueur_max + 1):
        for combo in itertools.product(charset, repeat=longueur):
            mot = ''.join(combo)
            tentatives += 1
            if hashlib.sha256(mot.encode()).hexdigest() == hash_cible:
                print(f'Trouvé : {mot} en {tentatives} tentatives')
                return mot

# Le script SA5_01 inclut un affichage de progression en temps réel.
```

Exercice pratique

1. Créez une wordlist avec Crunch (mots 3-4 chars, a-z)
2. Hashé en SHA-256 un mot court de votre choix
3. Lancez le script SA5_01 en mode dictionnaire puis bruteforce
4. Comparez les temps d'exécution

Méthode	Efficacité	Vitesse	Contre-mesure
Dictionnaire	Haute si mdp commun	Rapide	Mots de passe complexes uniques
Bruteforce	Totale (si assez de temps)	Très lente	Mots de passe longs + lockout
Rainbow Table	Haute (pré-calculé)	Instantanée	Salt aléatoire par utilisateur
Credential Stuffing	Moyenne	Dépend de l'API	MFA, détection d'anomalies

2.3 Contre-mesures et bonnes pratiques

- Utiliser bcrypt, Argon2 ou PBKDF2 plutôt que SHA-256 pour les mots de passe
- Ajouter un salt unique par utilisateur
- Limiter les tentatives de connexion (account lockout)
- Implémenter le MFA (Multi-Factor Authentication)
- Surveiller les anomalies de connexion (SIEM)
- Imposer des mots de passe longs (12+ caractères) avec complexité



Questions

- Q1. Quelle est la différence entre une Rainbow Table et une attaque bruteforce ?
- Q2. Expliquez pourquoi le salt rend les rainbow tables inefficaces.
- Q3. Combien de tentatives faut-il pour bruteforcer un mot de passe de 8 chars (a-z+0-9) ?
- Q4. Testez le script SA5_01 sur le hash SHA-256 de 'abc'. Quel temps faut-il en bruteforce ?
- Q5. Proposez 3 contre-mesures implémentables côté serveur contre le bruteforce.

Lab 3 — Keylogger

⚠️ AVERTISSEMENT LÉGAL ET ÉTHIQUE

Un keylogger est un outil qui enregistre les frappes clavier.
Son utilisation sans consentement est un crime dans la plupart des pays.
Ce lab se fait UNIQUEMENT sur votre propre machine de lab.
L'objectif est de comprendre comment ces outils fonctionnent pour mieux s'en protéger.

3.1 Principe de fonctionnement

Un keylogger intercepte les événements clavier à bas niveau (hooks système). Il peut opérer en mode utilisateur ou en mode kernel (plus furtif).

Type de keylogger	Niveau	Furtivité	Détection
Logiciel (user-space)	Application	Faible	Antivirus, EDR
Logiciel (kernel-level)	Noyau OS	Haute	Antirootkit, analyse mémoire
Hardware	Physique	Très haute	Inspection physique
Hyperviseur	VM/BIOS	Maximale	Très difficile

3.2 Implémentation Python — Script SA5_02

Le script **SA5_02_keylogger.py** utilise la bibliothèque `pynput` pour intercepter les événements clavier.

⚠️ Package requis

`pip install pynput --break-system-packages`
Le script installe automatiquement `pynput` au démarrage.

```
from pynput import keyboard

LOG_FILE = 'keylog.txt'

def on_press(key):
    try:
        char = key.char
    except AttributeError:
        char = f'[{key.name}]' # Touches spéciales

    with open(LOG_FILE, 'a') as f:
        f.write(char)

    if key == keyboard.Key.esc:
        return False # Arrêt sur Echap

with keyboard.Listener(on_press=on_press) as listener:
    listener.join()
```

Exercice pratique

1. Ouvrez le script SA5_02_keylogger.py dans VSCode
2. Exécutez-le, sélectionnez le mode 1 ou 2
3. Tapez quelques mots dans n'importe quelle fenêtre
4. Appuyez sur ECHAP pour arrêter
5. Lisez le fichier keylog_pedagogique.txt — observez les touches capturées
6. Ouvrez un gestionnaire de processus : top / htop — trouvez le processus Python

3.3 Contre-mesures

- Clavier virtuel (on-screen keyboard) pour les mots de passe sensibles
- Gestionnaire de mots de passe avec auto-remplissage (évite la frappe)
- Antivirus / EDR avec détection comportementale
- Authentification multi-facteurs (MFA) — même avec le mdp capturé, l'attaquant est bloqué
- Mise à jour régulière du système (patching des vulnérabilités kernel)
- Inspection physique des périphériques (contre les keyloggers hardware)



Questions

- Q1. Quelle bibliothèque Python permet de capturer les événements clavier ? Comment fonctionne-t-elle ?
- Q2. Listez les touches capturées dans votre fichier de log — distinguez lettres et touches spéciales.
- Q3. Comment un antivirus peut-il détecter un keylogger logiciel ?
- Q4. Proposez une architecture défensive contre les keyloggers pour un système bancaire.

Lab 4 — Ransomware (Simulation Contrôlée)

⚠️ AVERTISSEMENT LÉGAL ET ÉTHIQUE

Ce lab simule un ransomware dans un répertoire sandbox isolé (./sandbox_ransomware/).
Le script ne touche AUCUN fichier en dehors de ce répertoire.
Le déploiement d'un vrai ransomware est un crime grave, peine jusqu'à 7 ans d'emprisonnement + 300 000 € d'amende.
Travaillez UNIQUEMENT dans la VM de lab.

4.1 Fonctionnement d'un ransomware réel

Un ransomware suit généralement les étapes suivantes :

1. Infection : phishing, exploit, drive-by download, RDP brute-force
2. Reconnaissance : énumération des fichiers et des partages réseau
3. Communication C2 : génération et envoi de la clé au serveur attaquant
4. Chiffrement : AES pour les fichiers, RSA pour chiffrer la clé AES
5. Persistance : modification du registre, tâches planifiées
6. Notification : affichage de la note de rançon
7. (Optionnel) Exfiltration : vol des données avant chiffrement (double extorsion)

Ransomware célèbre	Année	Algorithme	Impact
WannaCry	2017	AES-128 + RSA-2048	150 pays, 300M\$
NotPetya	2017	AES-128 + RSA-2048	10 Md\$ de dégâts
REvil/Sodinokibi	2019-2021	AES-256 + ECC	Kaseya, JBS Foods
LockBit 3.0	2022-2024	AES + RSA-4096	Nombreuses entreprises

4.2 Architecture de la simulation

🔒 Sandbox isolée — Périmètre du script

Le script SA5_03_ransomware_pedagogique.py opère UNIQUEMENT dans :
./sandbox_ransomware/ ← répertoire créé par le script

Il crée des fichiers de test, les chiffre avec AES-256-CBC, puis peut les déchiffrer si la clé est disponible.

La clé est stockée localement (simulation) — dans un vrai ransomware, elle serait envoyée au C2 de l'attaquant, rendant le déchiffrement impossible.

4.3 Implémentation — Script SA5_03

Le script SA5_03_ransomware_pedagogique.py est interactif et propose un menu guidé.

⚠ Package requis

pip install pycryptodome --break-system-packages
 Le script installe automatiquement pycryptodome au démarrage.

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

def chiffrer_fichier(chemin, cle, iv):
    with open(chemin, 'rb') as f:
        data = f.read()
    cipher = AES.new(cle, AES.MODE_CBC, iv)
    encrypted = cipher.encrypt(pad(data, AES.block_size))
    chemin_locked = chemin + '.locked'
    with open(chemin_locked, 'wb') as f:
        f.write(encrypted)
    os.remove(chemin) # Supprime l'original
    return chemin_locked
```

Étapes du lab

1. Ouvrez SA5_03_ransomware_pedagogique.py dans VSCode et lisez le code
2. Exécutez le script → option 1 : Créer la sandbox
3. Option 2 : Listez les fichiers créés
4. Option 3 : Simulez le chiffrement — observez les fichiers .locked
5. Lisez la note de rançon générée dans la sandbox
6. Option 4 : Déchiffrez les fichiers — vérifiez la restauration
7. Option 5 : Nettoyez la sandbox

4.4 Contre-mesures et réponse aux ransomwares**Prévention**

- Règle de sauvegarde 3-2-1 : 3 copies, 2 supports différents, 1 hors site
- Principe de moindre privilège — n'accordez que les droits nécessaires
- Segmentation réseau — limitez les mouvements latéraux
- Patch management — appliquez les mises à jour de sécurité rapidement
- Formation des utilisateurs — résistance au phishing

Détection

- EDR (Endpoint Detection & Response) — analyse comportementale
- Honeypots et fichiers-leurres — détection précoce
- Surveillance des partages réseau — activité anormale de chiffrement
- SIEM — corrélation des événements suspects

Réponse (CSIRT)

- Isolation immédiate des systèmes infectés (déconnecter du réseau)
- Préservation des preuves (forensique)

- Identification du variant (ID Ransomware : id-ransomware.malwarehunterteam.com)
- Restauration depuis les sauvegardes — NE PAS PAYER la rançon
- Notification aux autorités (ANSSI, CNIL si données personnelles)



Questions

- Q1. Quel algorithme de chiffrement la simulation utilise-t-elle ? Pourquoi AES plutôt que RSA ?
- Q2. Observez le fichier `ransomware_key.json` — pourquoi cette clé ne serait-elle pas disponible dans un vrai ransomware ?
- Q3. Expliquez le concept de 'double extorsion' dans les ransomwares modernes.
- Q4. Proposez une architecture de sauvegarde résistante aux ransomwares (règle 3-2-1).
- Q5. Quelles actions immédiates prendre si vous détectez un ransomware actif sur votre réseau ?

5. Synthèse Comparative des Attaques

Attaque	Cible	Complexité impl.	Impact	Contre-mesure principale
Dictionnaire	Mots de passe hashés	Faible	Élevé (mdp communs)	bcrypt + salt + lockout
Bruteforce	Mots de passe courts	Faible	Moyen (lent)	Mots de passe longs + lockout
Keylogger	Données saisies	Moyenne	Très élevé	MFA + antivirus + clavier virtuel
Ransomware	Fichiers / systèmes	Haute	Catastrophique	Sauvegardes 3-2-1 + EDR

Questions de synthèse

Questions

- Q1. Comparez les attaques par dictionnaire et bruteforce : dans quel cas choisir l'une ou l'autre ?
- Q2. Comment un attaquant combine-t-il plusieurs techniques (wordlist + bruteforce + keylogger) ?
- Q3. Qu'est-ce que la Kill Chain d'une attaque ransomware ? Décrivez chaque phase.
- Q4. Proposez un plan de sécurité défensif complet pour une PME contre ces 4 types d'attaques.
- Q5. Quelle est la différence entre un ransomware 'locker' et un ransomware 'crypto' ?

6. Conclusion

Ce SA vous a permis d'explorer les techniques offensives fondamentales :

- Création de wordlists (Crunch, CeWL) : bases de toute attaque par dictionnaire
- Bruteforce et dictionnaire : compréhension des limites et protections
- Keylogger : interception des frappes et contre-mesures associées
- Ransomware : mécanisme de chiffrement malveillant et réponse à incident

La connaissance de ces techniques est indispensable pour les professionnels de la cybersécurité dans les domaines :

- Pentest et Red Team : simulation d'attaquants pour évaluer les défenses
- Blue Team et SOC : détection et réponse aux incidents
- Forensique : analyse post-incident
- DevSecOps : intégration de la sécurité dès la conception

 **Bon travail ! — S.Y. ♣**

EFREI Paris Panthéon-Assas Université
Usage pédagogique uniquement — Yousfi Souheib